

CEN3024C Module 4 Project

For this project, we will extend the Module 3 Task Manager application to introduce multiple entities, DTOs, and Hibernate associations. This will build on your knowledge of Spring Boot, JPA, and PostgreSQL by moving from the single-entity application to a multi-entity system with Users, Subtasks, and Priorities.

We are using a flattened representation for all DTOs so that API responses stay clean and lightweight. For example, the TaskDTO only lists subtask titles and assigned usernames instead of embedding full subtask or user objects, the UserDTO only lists task IDs instead of entire task details, and the SubtaskDTO points back to its parent task by ID. This design keeps responses easy to serialize, avoids circular references, and makes the data more predictable for clients. When more detail is needed, the client can make additional API calls (like GET /tasks/{id} or GET /users/{id}), which keeps responsibilities clear. As you work through the source code examples below, note that a separate DTO isn't provided for the Priority entity because the priorities are simple, fixed values represented by an enum (LOW, MEDIUM, HIGH). Unlike tasks, users, or subtasks, there's no sensitive data to hide or nested structure to flatten.

Objectives

- Add new entities (User, Subtask, Priority) and appropriate enumerations (PriorityLevel, SubtaskStatus) and update the DB schema.
- Define Hibernate associations (@OneToOne, @ManyToOne, @ManyToMany) between entities.
- Implement DTO classes for Task, User, and Subtask.
- Expand controllers, services, and repositories to support new entities.
- Modify/create cURL scripts to test new schema.

Using the starter code in this repository, complete the following:

1. Create New Entities

- User: represents system users.
- Subtask: represents smaller tasks linked to a Task.
- Priority: linked one-to-one with a task, representing its priority level.
- Add enums:
 - PriorityLevel (e.g., LOW, MEDIUM, HIGH)
 - SubtaskStatus (e.g., PENDING, IN_PROGRESS, COMPLETED)

2. Add Associations

- Task - User: many-to-many (a task can have multiple users, and a user can have multiple tasks).
- Task - Subtask: one-to-many (a task can have multiple subtasks).
- Task - Priority: one-to-one (each task has one priority).
- Use @JoinColumn and @JoinTable where appropriate.

3. Introduce a DTO Layer

- Create DTOs for Task, User, and Subtask in a new dto subpackage.
- DTOs should only expose necessary fields (e.g., exclude sensitive user data). In this project, this means:
 - User ID, password and any role/permission flags not needed by the UI.
 - raw database IDs, audit fields, or hidden system flags.
- Implement conversion methods in services to map between entities and DTOs.

4. Expand Repositories and Services

- Create repositories for User and Subtask.
- Add services for each new entity (UserService, SubtaskService), implementing CRUD operations.
- Update TaskService to handle subtasks, users, and priority.

5. Add Controllers

- Add UserController and SubtaskController with REST endpoints.
- Update TaskController to return DTOs and handle associations.

6. Database Updates

- Update schema with new tables (user, subtask, priority) and relationships.
- Use `spring.jpa.hibernate.ddl-auto=update` during development to apply schema changes automatically.

7. Testing

- Extend/create cURL test scripts to validate endpoints and demonstrate CRUD operations across all entities.
- Show that associations are working correctly (e.g., assigning subtasks to tasks, linking tasks to users).

Submission

- Upload your updated project to this assignment repository (**submission must be in the correct project format; do not create a new/separate project folder or zip the project**).
- Include updated screenshots showing API endpoints for Users, Subtasks, Tasks, and their relationships.
- Clearly label each screenshot with the corresponding endpoint tested.

Source Code

[resources/application.properties](#)

```
# application.properties
spring.application.name=M4Project

# Recreate schema on each run at startup
spring.jpa.hibernate.ddl-auto=create

# Disable Open Session in View
```

```
spring.jpa.open-in-view=false
```

```
# PostgreSQL connection
```

```
spring.datasource.url=jdbc:postgresql://localhost:5432/taskmanager_db
```

```
spring.datasource.username=postgres
```

```
spring.datasource.password=postgres
```

```
# Ensure data.sql is always executed (even with JPA/Hibernate)
```

```
spring.sql.init.mode=always
```

```
spring.jpa.defer-datasource-initialization=true
```

data.sql

```
INSERT INTO priorities (level) VALUES ('LOW') ON CONFLICT DO NOTHING;  
INSERT INTO priorities (level) VALUES ('MEDIUM') ON CONFLICT DO NOTHING;  
INSERT INTO priorities (level) VALUES ('HIGH') ON CONFLICT DO NOTHING;
```

entities/Task.java

```
// Task.java
```

```
// D. Singletary
```

```
// 9/10/25
```

```
// Task entity for task manager application
```

```
package edu.fscj.cen3024c.taskmanager.entities;
```

```
import jakarta.persistence.*;
```

```
import java.util.List;
```

```
import java.util.Set;
```

```
@Entity
```

```
@Table(name = "tasks")
```

```
public class Task {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Integer id;
```

```
    @Column(nullable = false)
```

```
    private String title;
```

```
    private String description;
```

```
    @Column(nullable = false, length = 20)
```

```
    private String status; // Example: PENDING, IN_PROGRESS, COMPLETED
```

```
    @Column(name = "due_date", length = 10)
```

```
    private String dueDate; // Format: YYYY-MM-DD
```

```

@ManyToOne
@JoinColumn(name = "priority_id")
private Priority priority;

// One-to-Many with Subtask
@OneToMany(mappedBy = "task", cascade = CascadeType.ALL, orphanRemoval =
true)
private List<Subtask> subtasks;

// Many-to-Many with User
@ManyToMany
@JoinTable(
    name = "user_tasks",
    joinColumns = @JoinColumn(name = "task_id"),
    inverseJoinColumns = @JoinColumn(name = "user_id")
)
private Set<User> users;

// Constructors
public Task() {}

public Task(String title, String description, String status, String
dueDate) {
    this.title = title;
    this.description = description;
    this.status = status;
    this.dueDate = dueDate;
}

// Getters & setters
public Integer getId() { return id; }
public void setId(Integer id) { this.id = id; }

public String getTitle() { return title; }
public void setTitle(String title) { this.title = title; }

public String getDescription() { return description; }
public void setDescription(String description) { this.description =
description; }

public String getStatus() { return status; }
public void setStatus(String status) { this.status = status; }

public String getDueDate() { return dueDate; }
public void setDueDate(String dueDate) { this.dueDate = dueDate; }

public Priority getPriority() { return priority; }
public void setPriority(Priority priority) { this.priority = priority; }

```

```

        public List<Subtask> getSubtasks() { return subtasks; }
        public void setSubtasks(List<Subtask> subtasks) { this.subtasks =
subtasks; }

        public Set<User> getUsers() { return users; }
        public void setUsers(Set<User> users) { this.users = users; }
    }

```

entities/Subtask.java

```

// Subtask.java
// D. Singletary
// 9/10/25
// Subtask entity for task manager application

package edu.fscj.cen3024c.taskmanager.entities;

import edu.fscj.cen3024c.taskmanager.enums.SubtaskStatus;
import jakarta.persistence.*;

@Entity
@Table(name = "subtasks")
public class Subtask {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(nullable = false)
    private String title;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private SubtaskStatus status = SubtaskStatus.PENDING; // default

    @ManyToOne
    @JoinColumn(name = "task_id", nullable = false)
    private Task task;

    // Constructors
    public Subtask() {}

    public Subtask(String title, SubtaskStatus status, Task task) {
        this.title = title;
        this.status = status;
        this.task = task;
    }

    // Getters & setters

```

```

    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public SubtaskStatus getStatus() { return status; }
    public void setStatus(SubtaskStatus status) { this.status = status; }

    public Task getTask() { return task; }
    public void setTask(Task task) { this.task = task; }
}

```

entities/User.java

```

// User.java
// D. Singletary
// 9/10/25
// User entity for task manager application

package edu.fscj.cen3024c.taskmanager.entities;

import jakarta.persistence.*;
import java.util.Set;

@Entity
@Table(name = "users") // avoid 'user' in postgresSQL
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String password;

    @ManyToMany(mappedBy = "users")
    private Set<Task> tasks;

    // Constructors
    public User() {}

    public User(String username, String password) {
        this.username = username;
        this.password = password;
    }
}

```

```

// Getters & setters
public Integer getId() { return id; }
public void setId(Integer id) { this.id = id; }

public String getUsername() { return username; }
public void setUsername(String username) { this.username = username; }

public String getPassword() { return password; }
public void setPassword(String password) { this.password = password; }

public Set<Task> getTasks() { return tasks; }
public void setTasks(Set<Task> tasks) { this.tasks = tasks; }
}

```

entities/Priority.java

```

// Priority.java
// D. Singletary
// 9/10/25
// Priority entity for task manager application

package edu.fscj.cen3024c.taskmanager.entities;

import com.fasterxml.jackson.annotation.JsonIgnore;
import edu.fscj.cen3024c.taskmanager.enums.PriorityLevel;
import jakarta.persistence.*;
import java.util.Set;

@Entity
@Table(name = "priorities")
public class Priority {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, unique = true)
    private PriorityLevel level;

    @OneToMany(mappedBy = "priority", fetch = FetchType.LAZY)
    @JsonIgnore // add this to stop JSON infinite recursion / lazy load issue
    private Set<Task> tasks;

    // Constructors
    public Priority() {}

    public Priority(PriorityLevel level) {

```

```

        this.level = level;
    }

    // Getters & setters
    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }

    public PriorityLevel getLevel() { return level; }
    public void setLevel(PriorityLevel level) { this.level = level; }

    public Set<Task> getTasks() { return tasks; }
    public void setTasks(Set<Task> tasks) { this.tasks = tasks; }
}

```

- Enums for the subtask status and priorities:

[enums/SubtaskStatus.java](#)

```

// SubtaskStatus.java
// D. Singletary
// 9/10/25
// Subtask enums for task manager application

package edu.fscj.cen3024c.taskmanager.enums;

public enum SubtaskStatus {
    PENDING,
    IN_PROGRESS,
    COMPLETED
}

```

[enums/PriorityLevel.java](#)

```

// PriorityLevel.java
// D. Singletary
// 9/10/25
// Priority enums for task manager application

package edu.fscj.cen3024c.taskmanager.enums;

public enum PriorityLevel {
    LOW,
    MEDIUM,
    HIGH
}

```

- Create a repository interface for each of the above entities in the repositories subpackage. Here is an example for the Task entity:

repositories/TaskRepository.java

```
// TaskRepository.java
// D. Singletary
// 9/10/25
// Repository for Task entity

package edu.fscj.cen3024c.taskmanager.repositories;

import edu.fscj.cen3024c.taskmanager.entities.Task;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface TaskRepository extends JpaRepository<Task, Integer> {
}
```

dto/TaskDTO.java

```
// TaskDTO.java
// D. Singletary
// 9/10/25
// Task DTO for task manager application

package edu.fscj.cen3024c.taskmanager.dto;

import java.util.List;
import java.util.Set;

public class TaskDTO {

    private Integer id;           // include task id
    private String title;
    private String description;
    private String status;
    private String dueDate;

    // Expose only what's useful for the client
    private String priorityLevel; // from Priority entity
    private List<String> subtasks; // titles of subtasks
    private Set<String> users;    // usernames of assigned users

    // Constructors
    public TaskDTO() {}
}
```

```

    public TaskDTO(Integer id, String title, String description, String
status, String dueDate,
                    String priorityLevel, List<String> subtasks, Set<String>
users) {
        this.id = id;
        this.title = title;
        this.description = description;
        this.status = status;
        this.dueDate = dueDate;
        this.priorityLevel = priorityLevel;
        this.subtasks = subtasks;
        this.users = users;
    }

    // Getters & setters
    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description =
description; }

    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }

    public String getDueDate() { return dueDate; }
    public void setDueDate(String dueDate) { this.dueDate = dueDate; }

    public String getPriorityLevel() { return priorityLevel; }
    public void setPriorityLevel(String priorityLevel) { this.priorityLevel =
priorityLevel; }

    public List<String> getSubtasks() { return subtasks; }
    public void setSubtasks(List<String> subtasks) { this.subtasks =
subtasks; }

    public Set<String> getUsers() { return users; }
    public void setUsers(Set<String> users) { this.users = users; }
}

```

dto/SubtaskDTO.java

```

// SubtaskDTO.java
// D. Singletary
// 9/10/25
// Subtask DTO for task manager application

```

```

package edu.fscj.cen3024c.taskmanager.dto;

public class SubtaskDTO {

    private Integer id;
    private String title;
    private String status;    // from SubtaskStatus enum
    private Integer taskId;   // parent Task ID

    // Constructors
    public SubtaskDTO() {}

    public SubtaskDTO(Integer id, String title, String status, Integer
taskId) {
        this.id = id;
        this.title = title;
        this.status = status;
        this.taskId = taskId;
    }

    // Getters & setters
    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }

    public Integer getTaskId() { return taskId; }
    public void setTaskId(Integer taskId) { this.taskId = taskId; }
}

```

dto/UserDTO.java

```

// UserDTO.java
// D. Singletary
// 9/10/25
// DTO for User entity (excludes password)

package edu.fscj.cen3024c.taskmanager.dto;

public class UserDTO {

    private Integer id;
    private String username;

```

```

// Constructors
public UserDTO() {}

public UserDTO(Integer id, String username) {
    this.id = id;
    this.username = username;
}

// Getters & setters
public Integer getId() { return id; }
public void setId(Integer id) { this.id = id; }

public String getUsername() { return username; }
public void setUsername(String username) { this.username = username; }
}

```

services/TaskService.java

```

// TaskService.java
// D. Singletary
// 9/10/25
// Task service for task manager application

package edu.fscj.cen3024c.taskmanager.services;

import edu.fscj.cen3024c.taskmanager.dto.TaskDTO;
import edu.fscj.cen3024c.taskmanager.entities.Subtask;
import edu.fscj.cen3024c.taskmanager.entities.Task;
import edu.fscj.cen3024c.taskmanager.entities.User;
import edu.fscj.cen3024c.taskmanager.exceptions.TaskNotFoundException;
import edu.fscj.cen3024c.taskmanager.repositories.TaskRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

@Service
public class TaskService {

    private final TaskRepository taskRepository;

    public TaskService(TaskRepository taskRepository) {
        this.taskRepository = taskRepository;
    }

    // CRUD Methods with DTO conversion

```

```

@Transactional(readOnly = true)
public List<TaskDTO> findAll() {
    return taskRepository.findAll()
        .stream()
        .map(this::convertToDTO)
        .collect(Collectors.toList());
}

@Transactional(readOnly = true)
public TaskDTO findById(Integer id) {
    Task task = taskRepository.findById(id)
        .orElseThrow(() -> new TaskNotFoundException(id));
    return convertToDTO(task);
}

// Helper to return raw entity (for deletes or internal use)
@Transactional(readOnly = true)
public Task findByIdEntity(Integer id) {
    return taskRepository.findById(id)
        .orElseThrow(() -> new TaskNotFoundException(id));
}

public Task save(Task task) {
    return taskRepository.save(task);
}

@Transactional
public TaskDTO updateTask(Integer id, Task taskDetails) {
    Task existingTask = findByIdEntity(id);
    existingTask.setTitle(taskDetails.getTitle());
    existingTask.setDescription(taskDetails.getDescription());
    existingTask.setStatus(taskDetails.getStatus());
    existingTask.setDueDate(taskDetails.getDueDate());
    existingTask.setPriority(taskDetails.getPriority());
    Task updatedTask = taskRepository.save(existingTask);
    return convertToDTO(updatedTask);
}

public void deleteById(Integer id) {
    if (!taskRepository.existsById(id)) {
        throw new TaskNotFoundException(id);
    }
    taskRepository.deleteById(id);
}

// Entity to DTO Conversion

public TaskDTO convertToDTO(Task task) {
    String priorityLevel = (task.getPriority() != null)

```

```

        ? task.getPriority().getLevel().name()
        : null;

    List<String> subtaskTitles = (task.getSubtasks() != null)
        ? task.getSubtasks().stream()
        .map(Subtask::getTitle)
        .collect(Collectors.toList())
        : List.of();

    Set<String> usernames = (task.getUsers() != null)
        ? task.getUsers().stream()
        .map(User::getUsername)
        .collect(Collectors.toSet())
        : Set.of();

    return new TaskDTO(
        task.getId(), // include id
        task.getTitle(),
        task.getDescription(),
        task.getStatus(),
        task.getDueDate(),
        priorityLevel,
        subtaskTitles,
        usernames
    );
}
}

```

services/SubtaskService.java

```

// SubtaskService.java
// D. Singletary
// 9/10/25
// Subtask service for task manager application

package edu.fscj.cen3024c.taskmanager.services;

import edu.fscj.cen3024c.taskmanager.dto.SubtaskDTO;
import edu.fscj.cen3024c.taskmanager.entities.Subtask;
import edu.fscj.cen3024c.taskmanager.entities.Task;
import edu.fscj.cen3024c.taskmanager.enums.SubtaskStatus;
import edu.fscj.cen3024c.taskmanager.exceptions.SubtaskNotFoundException;
import edu.fscj.cen3024c.taskmanager.repositories.SubtaskRepository;
import edu.fscj.cen3024c.taskmanager.repositories.TaskRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.stream.Collectors;

```

```

@Service
public class SubtaskService {

    private final SubtaskRepository subtaskRepository;
    private final TaskRepository taskRepository;

    public SubtaskService(SubtaskRepository subtaskRepository, TaskRepository
taskRepository) {
        this.subtaskRepository = subtaskRepository;
        this.taskRepository = taskRepository;
    }

    // CRUD Methods with DTO conversion

    @Transactional(readOnly = true)
    public List<SubtaskDTO> findAll() {
        return subtaskRepository.findAll()
            .stream()
            .map(this::convertToDTO)
            .collect(Collectors.toList());
    }

    @Transactional(readOnly = true)
    public SubtaskDTO findById(Integer id) {
        Subtask subtask = subtaskRepository.findById(id)
            .orElseThrow(() -> new SubtaskNotFoundException(id));
        return convertToDTO(subtask);
    }

    public Subtask save(Subtask subtask) {
        // Default status if not provided
        if (subtask.getStatus() == null) {
            subtask.setStatus(SubtaskStatus.PENDING);
        }
        return subtaskRepository.save(subtask);
    }

    @Transactional
    public SubtaskDTO updateSubtask(Integer id, Subtask subtaskDetails) {
        Subtask existingSubtask = subtaskRepository.findById(id)
            .orElseThrow(() -> new SubtaskNotFoundException(id));

        existingSubtask.setTitle(subtaskDetails.getTitle());
        existingSubtask.setStatus(subtaskDetails.getStatus());
        Subtask updatedSubtask = subtaskRepository.save(existingSubtask);

        return convertToDTO(updatedSubtask);
    }
}

```

```

    public void deleteById(Integer id) {
        if (!subtaskRepository.existsById(id)) {
            throw new SubtaskNotFoundException(id);
        }
        subtaskRepository.deleteById(id);
    }

    // Entity to DTO Conversion

    public SubtaskDTO convertToDTO(Subtask subtask) {
        Integer taskId = (subtask.getTask() != null) ?
subtask.getTask().getId() : null;

        return new SubtaskDTO(
            subtask.getId(),
            subtask.getTitle(),
            subtask.getStatus().name(),
            taskId
        );
    }
}

```

services/UserService.java

```

// UserService.java
// D. Singletary
// 9/10/25
// User service for task manager application

package edu.fscj.cen3024c.taskmanager.services;

import edu.fscj.cen3024c.taskmanager.dto.UserDTO;
import edu.fscj.cen3024c.taskmanager.entities.User;
import edu.fscj.cen3024c.taskmanager.exceptions.UserNotFoundException;
import edu.fscj.cen3024c.taskmanager.repositories.UserRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.stream.Collectors;

@Service
public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
}

```

```

    }

    // CRUD Methods with DTO conversion

    @Transactional(readOnly = true)
    public List<UserDTO> findAll() {
        return userRepository.findAll()
            .stream()
            .map(this::convertToDTO)
            .collect(Collectors.toList());
    }

    @Transactional(readOnly = true)
    public UserDTO findById(Integer id) {
        User user = userRepository.findById(id)
            .orElseThrow(() -> new UserNotFoundException(id));
        return convertToDTO(user);
    }

    // Helper to return raw entity (for updates/deletes)
    @Transactional(readOnly = true)
    public User findByIdEntity(Integer id) {
        return userRepository.findById(id)
            .orElseThrow(() -> new UserNotFoundException(id));
    }

    public User save(User user) {
        return userRepository.save(user);
    }

    @Transactional
    public UserDTO updateUser(Integer id, User userDetails) {
        User existingUser = findByIdEntity(id);
        existingUser.setUsername(userDetails.getUsername());
        existingUser.setPassword(userDetails.getPassword()); // will be
hashed in a later module
        User updatedUser = userRepository.save(existingUser);
        return convertToDTO(updatedUser);
    }

    public void deleteById(Integer id) {
        if (!userRepository.existsById(id)) {
            throw new UserNotFoundException(id);
        }
        userRepository.deleteById(id);
    }

    // Entity to DTO Conversion

```

```

        public UserDTO convertToDTO(User user) {
            return new UserDTO(
                user.getId(),
                user.getUsername()
            );
        }
    }
}

```

services/PriorityService.java

```

// PriorityService.java
// D. Singletary
// 9/10/25
// Read-only service for Priority entity

package edu.fscj.cen3024c.taskmanager.services;

import edu.fscj.cen3024c.taskmanager.entities.Priority;
import edu.fscj.cen3024c.taskmanager.exceptions.PriorityNotFoundException;
import edu.fscj.cen3024c.taskmanager.repositories.PriorityRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
public class PriorityService {

    private final PriorityRepository priorityRepository;

    public PriorityService(PriorityRepository priorityRepository) {
        this.priorityRepository = priorityRepository;
    }

    @Transactional(readOnly = true)
    public List<Priority> findAll() {
        return priorityRepository.findAll();
    }

    @Transactional(readOnly = true)
    public Priority findById(Integer id) {
        return priorityRepository.findById(id)
            .orElseThrow(() -> new PriorityNotFoundException(id));
    }
}

```

- Create custom exception classes for “<entity>NotFound” exceptions. Here is an example for the Task entity:

exceptions/TaskNotFoundException.java

```
// TaskNotFoundException.java
// D. Singletary
// 9/10/25
// Exception for handling Task not found cases

package edu.fscj.cen3024c.taskmanager.exceptions;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.NOT_FOUND)
public class TaskNotFoundException extends RuntimeException{
    public TaskNotFoundException(Integer id) {
        super("Task not found with id " + id);
    }
}
```

controllers/TaskController.java

```
// TaskController.java
// D. Singletary
// 9/10/25
// Task controller for task manager application

package edu.fscj.cen3024c.taskmanager.controllers;

import edu.fscj.cen3024c.taskmanager.dto.TaskDTO;
import edu.fscj.cen3024c.taskmanager.entities.Priority;
import edu.fscj.cen3024c.taskmanager.entities.Task;
import edu.fscj.cen3024c.taskmanager.exceptions.PriorityNotFoundException;
import edu.fscj.cen3024c.taskmanager.repositories.PriorityRepository;
import edu.fscj.cen3024c.taskmanager.services.TaskService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/tasks")
public class TaskController {

    @Autowired
    private TaskService taskService;

    @Autowired
```

```

private PriorityRepository priorityRepository;

// READ endpoints return DTOs

@GetMapping
public List<TaskDTO> getAllTasks() {
    return taskService.findAll();
}

@GetMapping("/{id}")
public TaskDTO getTaskById(@PathVariable Integer id) {
    return taskService.findById(id);
}

// WRITE endpoints accept entities but return DTOs

@PostMapping
public TaskDTO createTask(@RequestBody Task task) {
    if (task.getPriority() != null && task.getPriority().getId() != null)
    {
        Priority priority =
priorityRepository.findById(task.getPriority().getId())
                .orElseThrow(() -> new
PriorityNotFoundException(task.getPriority().getId()));
        task.setPriority(priority);
    }
    Task savedTask = taskService.save(task);
    return taskService.convertToDTO(savedTask);
}

@PutMapping("/{id}")
public TaskDTO updateTask(@PathVariable Integer id, @RequestBody Task
task) {
    if (task.getPriority() != null && task.getPriority().getId() != null)
    {
        Priority priority=
priorityRepository.findById(task.getPriority().getId())
                .orElseThrow(() -> new
PriorityNotFoundException(task.getPriority().getId()));
        task.setPriority(priority);
    }
    return taskService.updateTask(id, task);
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteTask(@PathVariable Integer id) {
    taskService.deleteById(id);
    return ResponseEntity.noContent().build();
}

```

```
    }  
}
```

controllers/SubtaskController.java

```
// SubtaskController.java  
// D. Singletary  
// 9/10/25  
// Subtask controller for task manager application  
  
package edu.fscj.cen3024c.taskmanager.controllers;  
  
import edu.fscj.cen3024c.taskmanager.dto.SubtaskDTO;  
import edu.fscj.cen3024c.taskmanager.entities.Subtask;  
import edu.fscj.cen3024c.taskmanager.services.SubtaskService;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.*;  
  
import java.util.List;  
  
@RestController  
@RequestMapping("/subtasks")  
public class SubtaskController {  
  
    @Autowired  
    private SubtaskService subtaskService;  
  
    // READ endpoints return DTOs  
  
    @GetMapping  
    public List<SubtaskDTO> getAllSubtasks() {  
        return subtaskService.findAll();  
    }  
  
    @GetMapping("/{id}")  
    public SubtaskDTO getSubtaskById(@PathVariable Integer id) {  
        return subtaskService.findById(id);  
    }  
  
    // WRITE endpoints accept entities but return DTOs  
  
    @PostMapping  
    public SubtaskDTO createSubtask(@RequestBody Subtask subtask) {  
        Subtask savedSubtask = subtaskService.save(subtask);  
        return subtaskService.convertToDTO(savedSubtask);  
    }  
  
    @PutMapping("/{id}")
```

```

        public SubtaskDTO updateSubtask(@PathVariable Integer id, @RequestBody
Subtask subtask) {
            return subtaskService.updateSubtask(id, subtask);
        }

        @DeleteMapping("/{id}")
        public ResponseEntity<Void> deleteSubtask(@PathVariable Integer id) {
            subtaskService.deleteById(id);
            return ResponseEntity.noContent().build();
        }
    }
}

```

controllers/UserController.java

```

// UserController.java
// D. Singletary
// 9/10/25
// User controller for task manager application

package edu.fscj.cen3024c.taskmanager.controllers;

import edu.fscj.cen3024c.taskmanager.dto.UserDTO;
import edu.fscj.cen3024c.taskmanager.entities.User;
import edu.fscj.cen3024c.taskmanager.services.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/users")
public class UserController {

    @Autowired
    private UserService userService;

    // READ endpoints return DTOs

    @GetMapping
    public List<UserDTO> getAllUsers() {
        return userService.findAll();
    }

    @GetMapping("/{id}")
    public UserDTO getUserById(@PathVariable Integer id) {
        return userService.findById(id);
    }
}

```

```

// WRITE endpoints accept/return entities

@PostMapping
public User createUser(@RequestBody User user) {
    return userService.save(user);
}

@PutMapping("/{id}")
public UserDTO updateUser(@PathVariable Integer id, @RequestBody User
user) {
    return userService.updateUser(id, user);
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteUser(@PathVariable Integer id) {
    userService.deleteById(id);
    return ResponseEntity.noContent().build();
}
}

```

controllers/PriorityController.java

```

// PriorityController.java
// D. Singletary
// 9/10/25
// Controller for Priority entity (read-only)

package edu.fscj.cen3024c.taskmanager.controllers;

import edu.fscj.cen3024c.taskmanager.entities.Priority;
import edu.fscj.cen3024c.taskmanager.services.PriorityService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/priorities")
public class PriorityController {

    @Autowired
    private PriorityService priorityService;

    // READ ONLY

    @GetMapping
    public List<Priority> getAllPriorities() {
        return priorityService.findAll();
    }
}

```

```

    @GetMapping("/{id}")
    public Priority getPriorityById(@PathVariable Integer id) {
        return priorityService.findById(id);
    }
}

```

NOTES

- @JoinColumn(name = "priority_id") creates a foreign key in the tasks table pointing to the priority table. Many tasks can share the same priority (e.g., multiple tasks marked HIGH), so in Task.java, it is a @ManyToOne association.
- @JoinTable(name = "user_tasks", ...) implicitly creates join/association table for the many-to-many relationship between users and tasks.
- @OneToMany(mappedBy = "task") in Task pairs with a @ManyToOne in Subtask to form a one-to-many relationship.
- @Table(name = "users") avoids conflict with PostgreSQL's reserved word user.
- Unique username ensures no duplicate accounts.
- password column is present; this will be hashed in an upcoming module
- The bidirectional @ManyToMany mapping uses mappedBy = "users" in the User entity, which links to the @JoinTable defined in the Task entity. No second join table is created; User.tasks and Task.users both reference the same association table (user_tasks).
- Spring Boot will automatically run data.sql at startup against the datasource defined in application.properties

cURL commands

REM Create a new task

```

curl -X POST http://localhost:8080/tasks ^
-H "Content-Type: application/json" ^
-d '{"title": "New Task", "description": "Task description",
  "status": "PENDING", "dueDate": "2024-09-15"}'

```

REM Get all tasks

```

curl -X GET http://localhost:8080/tasks

```

REM Get task with ID 1

```

curl -X GET http://localhost:8080/tasks/1

```

REM Update task with ID 1

```

curl -X PUT http://localhost:8080/tasks/1 ^
-H "Content-Type: application/json" ^
-d '{"title": "Updated Task", "description": "Updated description",
  "status": "COMPLETED", "dueDate": "2024-10-01"}'

```

```

REM Delete task with ID 1
REM curl -X DELETE http://localhost:8080/tasks/1

REM Create a new user
curl -X POST http://localhost:8080/users ^
-H "Content-Type: application/json" ^
-d '{"username\: \"alice\", \"password\: \"password123\"}'"

REM Get all users (returns DTOs, no password field)
curl -X GET http://localhost:8080/users

REM Get a specific user by ID (replace 1 with actual ID if different)
curl -X GET http://localhost:8080/users/1

REM Update a user (returns DTO, still no password field)
curl -X PUT http://localhost:8080/users/1 ^
-H "Content-Type: application/json" ^
-d '{"username\: \"alice_updated\", \"password\: \"newpass456\"}'"

REM Delete a user
curl -X DELETE http://localhost:8080/users/1

REM Get all priorities (should return LOW, MEDIUM, HIGH from data.sql)
curl -X GET http://localhost:8080/priorities

REM Get priority with ID 1 (LOW)
curl -X GET http://localhost:8080/priorities/1

REM Get priority with ID 2 (MEDIUM)
curl -X GET http://localhost:8080/priorities/2

REM Get priority with ID 3 (HIGH)
curl -X GET http://localhost:8080/priorities/3

REM Create a new task with Priority HIGH (id = 3)
curl -X POST http://localhost:8080/tasks ^
-H "Content-Type: application/json" ^
-d '{ \"title\: \"Finish project\", \"description\: \"Complete milestone
1\", \"status\: \"PENDING\", \"dueDate\: \"2024-09-30\", \"priority\: {
\"id\: 3 } }"'

REM Create a new subtask for Task with ID 1 (status defaults to PENDING)
curl -X POST http://localhost:8080/subtasks ^
-H "Content-Type: application/json" ^
-d '{ \"title\: \"Write unit tests\", \"status\: \"PENDING\", \"task\: {
\"id\: 1 } }"'

REM Get all subtasks
curl -X GET http://localhost:8080/subtasks

```

REM Get subtask with ID 1

```
curl -X GET http://localhost:8080/subtasks/1
```

REM Update subtask with ID 1 (set status to COMPLETED)

```
curl -X PUT http://localhost:8080/subtasks/1 ^
```

```
-H "Content-Type: application/json" ^
```

```
-d "{ \"title\": \"Write unit tests\", \"status\": \"COMPLETED\", \"task\": {  
  \"id\": 1 } }"
```

REM Delete subtask with ID 1

```
curl -X DELETE http://localhost:8080/subtasks/1
```